

Modern CLI

gh · GitHub Copilot CLI

Overview

The modern developer's terminal is no longer just a place to run commands, it's a fully AI-augmented workspace. In this context, the objective is to build devkit, a personal CLI meta-tool that orchestrates GitHub CLI, GitHub Copilot CLI, and other modern tools into a seamless, opinionated developer workflow.

In addition to learning each tool in isolation, learn to integrate them, understand how they complement each other, and build scripted workflows that a real development team would actually use.

Project Goal

Build devkit — a Python-powered CLI meta-tool that wraps and orchestrates modern CLI tools. The final product automates common developer tasks: starting a feature, getting AI code review, generating commits, searching issues, and summarizing PR activity — all from one command.

The Modern CLI Ecosystem

Before diving in, understand what makes these tools 'modern' and how they differ from classic Unix tools.

Tool	What it does	Why it matters
gh	GitHub CLI — issues, PRs, repos from terminal	Replaces browser workflow entirely
gh copilot	AI command suggestions & explanations	Turns natural language into shell commands
gemini	Google's Gemini AI in your terminal	Multi-modal AI for code + docs analysis
claude	Anthropic Claude Code — agentic coding assistant	Edits files, runs commands, understands repos
warp	AI-native terminal (terminal emulator)	AI completions & workflows built into the shell
fzf	Fuzzy finder for any list	Makes every selection interactive
bat	cat replacement with syntax highlighting	Better code reading in terminal
delta	Better git diff viewer	Readable diffs with side-by-side view
lazygit	Terminal UI for git	Full git workflow without memorizing commands
atuin	Shell history with search + sync	Searchable, shareable command history

Objectives

- Understand the role and API surface of modern CLI tool
- Combine multiple CLI tools using Python's subprocess module
- Build a CLI orchestration layer with Typer that feels native
- Use AI CLI tools (Copilot, Gemini) as programmable tools, not just chat interfaces
- Design human-friendly and machine-friendly CLI output
- Apply real-world developer workflows: feature branches, PR review, commit generation

Phase	Topic
Phase 1	Tool Discovery & Setup
Phase 2	GitHub CLI
Phase 3	AI CLI Tools: Copilot, Gemini etc.
Phase 4	Building devkit - The Orchestration Layer
Phase 5	Polish, Packaging & Demo



PHASE 1

Tool Discovery & Setup

Overview

Install, authenticate, and explore every tool you will orchestrate. This phase is hands-on discovery: run each tool, read its `--help`, understand what it outputs and how.

Step 1 — Install the toolchain

```
# macOS (Homebrew)
brew install gh fzf bat git-delta lazygit atuin
brew install --cask warp

# GitHub Copilot CLI extension
gh auth login
gh extension install github/gh-copilot

# Gemini CLI
npm install -g @google/generative-ai-cli
# or: pip install google-generativeai (Python SDK)

# Claude Code
npm install -g @anthropic-ai/claude-code

# Python toolchain
pip install typer[all] rich subprocess-run
```

Step 2 — Authenticate each tool

Tool	Auth Command
gh	gh auth login (browser OAuth)
gh copilot	requires gh auth — inherits token
gemini	gemini auth (Google account)
claude	claude auth (Anthropic API key or browser)
atuin	atuin register (optional sync)

Step 3 — Discovery exercises

For each tool below, run the given commands and write 2–3 sentences describing what you observed. Answers go in a `discovery.md` file in the project repo.

```
# GitHub CLI — explore your repos
gh repo list --limit 10
gh issue list --repo cli/cli
gh pr list --repo cli/cli --state open
```

```
# Copilot CLI – AI command suggestions
gh copilot suggest 'find all python files modified in the last week'
gh copilot explain 'find . -name *.py -mtime -7 -exec wc -l {} +'

# bat – better cat
bat --list-themes
bat --theme=GitHub README.md

# delta – better diffs
git diff HEAD~1 | delta

# fzf – fuzzy finder
ls | fzf
git log --oneline | fzf

# Gemini CLI
gemini 'Explain what a Makefile does in 3 sentences'

# Claude Code
claude 'Summarize the purpose of this repository'
```

**Deliverable — Phase 1**

File: discovery.md committed to your project repo

For each tool: what it does, one surprising thing, one use case you want to try.

Create the project repo with: `gh repo create devkit --public --clone`



GitHub CLI Deep Dive

Overview

The GitHub CLI (gh) is the most powerful tool in the modern developer's terminal. In this phase, you learn its full surface area and wire it into Python scripts using subprocess, building the foundational commands that devkit will expose.

Core gh Command Groups

Command group	Key commands	Output format
gh repo	create, clone, fork, view	Text / browser
gh issue	create, list, view, close, comment	Table / JSON
gh pr	create, list, view, merge, diff	Table / JSON
gh release	create, list, view, upload	Table / JSON
gh run	list, view, watch, rerun	Table / JSON
gh gist	create, list, edit, delete	Text / URL
gh api	raw REST & GraphQL calls	Raw JSON
gh search	repos, issues, prs, code	Table / JSON

Step 4 — Structured JSON output

gh can output JSON for almost any command using `--json`. This is the key to using it programmatically from Python.

```
# List fields available for issues
gh issue list --json 2>&1 | head -5

# Get issues as JSON
gh issue list --json number,title,state,labels,assignees --limit 20

# Filter with jq
gh issue list --json number,title,state | jq '.[[] |
select(.state=="OPEN")'

# Get a single PR as JSON
gh pr view 42 --json title,body,reviews,files

# Search across GitHub
gh search issues 'is:open label:bug repo:myorg/myapp' --json
number,title,url
```

Step 5 — Calling gh from Python

```
# utils/gh.py
import subprocess
import json
from typing import Any

def gh(*args: str) -> str:
    '''Run a gh command and return stdout as string.'''
    result = subprocess.run(
        ['gh', *args],
        capture_output=True, text=True, check=True
    )
    return result.stdout.strip()

def gh_json(*args: str) -> Any:
    '''Run a gh command with --json and parse result.'''
    raw = gh(*args)
    return json.loads(raw) if raw else []

# Usage
issues = gh_json('issue', 'list', '--json', 'number,title,state', '--limit', '10')
for issue in issues:
    print(f"#{issue['number']} {issue['title']} [{issue['state']}]")
```

Step 6 — Build the gh module for devkit

Implement the following commands that devkit will expose. Each wraps one or more gh calls.

devkit command	What it does (under the hood)
<code>devkit issues</code>	gh issue list as a rich table, colored by state
<code>devkit pr-summary</code>	gh pr view + gh pr diff — shows PR title, body, changed files
<code>devkit start-feature</code>	gh repo fork + git checkout -b <branch>
<code>devkit open-pr</code>	gh pr create with interactive prompts for title and body
<code>devkit run-status</code>	gh run list — latest CI run status per branch

```
# Example: devkit issues command
@app.command()
def issues(
    repo: str = typer.Option('', help='owner/repo (default: current repo)'),
    limit: int = typer.Option(15, help='Max number of issues'),
):
    '''List open issues in a rich table.'''
    args = ['issue', 'list', '--json', 'number,title,state,labels', '--limit', str(limit)]
    if repo:
        args += ['--repo', repo]
    data = gh_json(*args)
```

```
table = Table(title='Open Issues', border_style='green')
table.add_column('#', style='cyan', width=6)
table.add_column('Title', min_width=30)
table.add_column('Labels', width=20)

for issue in data:
    labels = ', '.join(l['name'] for l in issue.get('labels', []))
    table.add_row(str(issue['number']), issue['title'], labels or
'-')

console.print(table)
```

**Deliverable — Phase 2**

File: devkit/commands/github.py with all 5 commands implemented.

Each command must use --json output and display a rich-formatted table.

Commit and push using: gh pr create (create your first PR with gh!)



AI CLI Tools

Overview

This phase covers the three AI CLI tools at the heart of the modern developer workflow: GitHub Copilot CLI for command suggestions, Gemini CLI for code and document analysis, and Claude Code for agentic repo-level tasks. Learn to use each as a programmable tool -- not a chatbot.

3.1 — GitHub Copilot CLI

gh copilot — Turns natural language into shell commands

Copilot CLI has two core commands: `suggest` and `explain`. `suggest` generates a shell command from a natural language description. `explain` breaks down what a complex command does.

```
# Suggest a command
gh copilot suggest 'list all docker containers sorted by memory usage'

# Explain an existing command
gh copilot explain 'awk NR==FNR{a[$1]=$2;next} $1 in a{print $0,a[$1]}'

# Use --target to specify shell type
gh copilot suggest --target shell 'compress all jpg files older than 30 days'
gh copilot suggest --target git 'undo my last commit but keep changes staged'
gh copilot suggest --target gh 'create a release from the latest tag'
```

Step 7 — Integrating Copilot CLI into devkit

Build a devkit `explain` command and a devkit `suggest` command that wrap Copilot CLI and format output with `rich`.

```
# commands/ai.py
import subprocess
from rich.panel import Panel

@app.command()
def explain(command: str = typer.Argument(..., help='Shell command to explain')):
    '''Ask Copilot CLI to explain a shell command.'''
    result = subprocess.run(
        ['gh', 'copilot', 'explain', command],
        capture_output=True, text=True
    )
    console.print(Panel(result.stdout, title='[purple]Copilot Explanation[/purple]', border_style='purple'))
```

```
@app.command()
def suggest(task: str = typer.Argument(..., help='Task to
accomplish')):
    '''Ask Copilot CLI to suggest a command.'''
    result = subprocess.run(
        ['gh', 'copilot', 'suggest', task],
        capture_output=True, text=True
    )
    console.print(Panel(result.stdout, title='[purple]Copilot
Suggestion[/purple]', border_style='purple'))
```

3.2 — Gemini CLI

gemini — Google Gemini AI — multi-modal, fast, generous free tier

Gemini CLI can analyze text, code, and images from the terminal. In devkit, it serves as the 'code review' and 'summarize' engine — reading files and producing structured feedback.

```
# Basic prompt
gemini 'What is the difference between async and threading in Python?'

# Pipe code to Gemini for review
cat utils/gh.py | gemini 'Review this Python code for bugs and
improvements'

# Summarize a PR diff
gh pr diff 42 | gemini 'Summarize these code changes in plain English'

# Read a file and ask about it
gemini --file README.md 'What setup steps are missing from this
README?'

# Generate a commit message from staged diff
git diff --staged | gemini 'Write a conventional commit message for
these changes'
```

Step 8 — The AI Review Pipeline

Build devkit review: a command that gets a PR diff from gh, pipes it to Gemini, and displays the feedback in a rich panel. This is the core integration of the project.

```
@app.command()
def review(
    pr_number: int = typer.Argument(..., help='PR number to review'),
    model: str = typer.Option('gemini', help='AI tool: gemini or
claude'),
):
    '''AI-powered code review of a pull request.'''
    with Progress(SpinnerColumn(), TextColumn('{task.description}')) as progress:
```

```

        t = progress.add_task('Fetching PR diff...')
        diff = gh('pr', 'diff', str(pr_number))
        pr_info = gh_json('pr', 'view', str(pr_number), '--json',
'title,body')

        progress.update(t, description='Running AI review...')
        prompt = f'Review this PR titled "{pr_info["title"]}":
\n\n{diff[:4000]}'

        if model == 'gemini':
            result = subprocess.run(['gemini', prompt],
capture_output=True, text=True)
        else:
            result = subprocess.run(['claude', prompt],
capture_output=True, text=True)

        console.print(Panel(result.stdout, title=f'[cyan]AI Review – PR
#{pr_number}[/cyan]', border_style='cyan'))

```

3.3 — Claude Code

claude — Anthropic Claude Code — agentic, repo-aware, file-editing AI

Claude Code is fundamentally different from Copilot and Gemini in the terminal: it is an agentic tool that reads your repo, edits files, runs commands, and takes multi-step actions. It understands your codebase as a whole.

Mode	Usage
One-shot prompt	claude 'Explain what this project does'
File context	claude --file src/main.py 'Find all bugs'
Interactive session	claude (launches interactive agent)
Non-interactive	claude --no-interactive 'Add type hints to all functions in utils.py'
With context	cat error.log claude 'Diagnose this error'

```

# One-shot from terminal
claude 'What are the main entry points of this codebase?'

# Pipe an error for diagnosis
python main.py 2>&1 | claude 'Explain this error and suggest a fix'

# Generate documentation
claude 'Write a CONTRIBUTING.md for this project based on the code
structure'

# Non-interactive editing
claude --no-interactive 'Add docstrings to every function in
utils/gh.py'

```

Step 9 — Smart Commit Message Generator

Build `devkit commit`: a command that reads the staged diff, sends it to Claude or Gemini, and proposes a conventional commit message. The user can accept, edit, or regenerate.

```
@app.command()
def commit():
    '''Generate a commit message from staged changes using AI.'''
    diff = subprocess.check_output(['git', 'diff', '--staged'],
    text=True)
    if not diff.strip():
        console.print('[yellow]No staged changes.[/yellow]')
        raise typer.Exit()

    prompt = f'Write a conventional commit message for these staged
changes:\n\n{diff[:3000]}'
    result = subprocess.run(['claude', '--no-interactive', prompt],
capture_output=True, text=True)
    suggested = result.stdout.strip()

    console.print(Panel(suggested, title='[green]Suggested Commit
Message[/green]'))
    use_it = Confirm.ask('Use this message?')
    if use_it:
        subprocess.run(['git', 'commit', '-m', suggested])
    else:
        manual = Prompt.ask('Enter your message')
        subprocess.run(['git', 'commit', '-m', manual])
```



Deliverable — Phase 3

File: `devkit/commands/ai.py` with: `explain`, `suggest`, `review`, `commit`

Each command must call an actual AI CLI tool as a subprocess.

Test: run `devkit review` on your own PR from Phase 2.



PHASE 4

Building devkit — The Orchestration Layer

Overview

Phases 2 and 3 produced isolated commands. Phase 4 assembles them into devkit: a cohesive, installable CLI tool with a unified interface, shared configuration, and a flagship workflow command that chains multiple tools together.

Step 10 — Project structure

```
devkit/
├── src/
│   ├── devkit/
│   │   ├── __init__.py
│   │   ├── main.py           # Root Typer app
│   │   ├── config.py        # Settings from ~/.devkit/config.json
│   │   ├── commands/
│   │   │   ├── github.py    # Phase 2 commands
│   │   │   ├── ai.py        # Phase 3 commands
│   │   │   └── workflow.py  # Phase 4 orchestration
│   │   └── utils/
│   │       ├── gh.py        # gh subprocess wrapper
│   │       ├── shell.py     # Generic subprocess helpers
│   │       └── display.py   # Shared rich formatters
│   ├── tests/
│   ├── pyproject.toml
│   └── README.md
```

Step 11 — Unified app with command groups

```
# main.py
import typer
from devkit.commands import github, ai, workflow

app = typer.Typer(
    name='devkit',
    help='AI-powered developer toolkit',
    rich_markup_mode='rich',
)

# Register sub-apps
app.add_typer(github.app, name='gh', help='GitHub operations')
app.add_typer(ai.app, name='ai', help='AI tools (Copilot, Gemini, Claude)')
app.add_typer(workflow.app, name='workflow', help='End-to-end dev workflows')

@app.callback(invoked_without_command=True)
def main(ctx: typer.Context):
    if ctx.invoked_subcommand is None:
```

```

        console.print(Panel('Welcome to [bold cyan]devkit[/bold cyan]',
border_style='cyan'))
        console.print(ctx.get_help())

if __name__ == '__main__':
    app()

```

Step 12 — The flagship workflow: devkit workflow feature-start

This command chains gh, git, and AI tools together into one complete action: create a branch, open a draft PR, and get an AI scaffold for the work.

```

@workflow_app.command('feature-start')
def feature_start(
    name: str = typer.Argument(..., help='Feature name (kebab-case)'),
    issue: int = typer.Option(None, help='Issue number to link'),
):
    '''Start a new feature: branch + draft PR + AI scaffold.'''

    console.rule('[bold]Starting Feature[/bold]')

    # 1. Create branch
    branch = f'feature/{name}'
    subprocess.run(['git', 'checkout', '-b', branch], check=True)
    console.print(f'[green]✓[/green] Created branch: {branch}')

    # 2. Push branch
    subprocess.run(['git', 'push', '-u', 'origin', branch], check=True)

    # 3. Create draft PR
    pr_title = name.replace('-', ' ').title()
    pr_args = ['pr', 'create', '--draft', '--title', pr_title, '--
body', '']
    if issue:
        pr_args += ['--body', f'Closes #{issue}']
    pr_url = gh(*pr_args)
    console.print(f'[green]✓[/green] Draft PR: {pr_url}')

    # 4. AI scaffold
    if issue:
        issue_body = gh_json('issue', 'view', str(issue), '--json',
'title,body')
        prompt = f'I\'m starting work on: {issue_body["title"]}
\n{issue_body["body"]}\nSuggest a step-by-step implementation plan.'
        plan = subprocess.run(['claude', '--no-interactive', prompt],
capture_output=True, text=True)
        console.print(Panel(plan.stdout, title='[cyan]AI Implementation
Plan[/cyan]', border_style='cyan'))

    console.rule('[green]Ready to code![/green]')

```

Step 13 — Configuration file

devkit should read user preferences from `~/.devkit/config.json` so you can set your preferred AI tool, default repo, and color theme.

```
# config.py
import json
from pathlib import Path

CONFIG_FILE = Path.home() / '.devkit' / 'config.json'

DEFAULTS = {
    'ai_tool': 'claude',          # or 'gemini'
    'default_repo': '',
    'theme': 'dark',
    'show_spinner': True,
}

def load_config() -> dict:
    if CONFIG_FILE.exists():
        return {**DEFAULTS, **json.loads(CONFIG_FILE.read_text())}
    return DEFAULTS

def save_config(cfg: dict):
    CONFIG_FILE.parent.mkdir(exist_ok=True)
    CONFIG_FILE.write_text(json.dumps(cfg, indent=2))
```

Step 14 — fzf integration for interactive selection

When listing issues or PRs, add `--interactive (-i)` flag that pipes results through `fzf` for fuzzy selection.

```
def fzf_select(items: list[str], prompt: str = 'Select > ') -> str:
    '''Pipe items through fzf and return selected line.'''
    proc = subprocess.run(
        ['fzf', f'--prompt={prompt}', '--height=40%', '--border'],
        input='\n'.join(items),
        capture_output=True, text=True
    )
    return proc.stdout.strip()

# Usage in issues command
if interactive:
    lines = [f"#{i['number']} {i['title']}" for i in data]
    selected = fzf_select(lines, prompt='Issue > ')
    issue_num = selected.split()[0].lstrip('#')
    gh('issue', 'view', issue_num, '--web')  # open in browser
```

Deliverable — Phase 4

Working devkit installable via: `pip install -e .`

Command groups: `devkit gh`, `devkit ai`, `devkit workflow`

Flagship: `devkit workflow feature-start <name> [--issue N]` must work end-to-end

Config: ~/.devkit/config.json respected by all AI commands



Overview

The final phase focuses on making devkit presentable, robust, and shareable. Polish error handling, write a proper README, create a demo recording, and optionally publish to PyPI or GitHub Releases.

Step 15 — Error handling for missing tools

devkit should check for each required tool at startup and give clear instructions if any are missing.

```
# utils/check.py
import shutil
from rich.console import Console

console = Console(stderr=True)

REQUIRED_TOOLS = {
    'gh':      'Install from https://cli.github.com',
    'fzf':     'brew install fzf or apt install fzf',
    'bat':     'brew install bat or apt install bat',
    'delta':   'brew install git-delta',
}

def check_tools():
    missing = {t: hint for t, hint in REQUIRED_TOOLS.items() if not
shutil.which(t)}
    if missing:
        console.print('[red]Missing required tools:[/red]')
        for tool, hint in missing.items():
            console.print(f'  [cyan]{tool}[/cyan] - {hint}')
        raise SystemExit(1)
```

Step 16 — README.md with real examples

The README should function as a demo — showing real command output. Use the script command or asciinema to capture real terminal output and embed it.

```
# Record terminal session with asciinema
asciinema rec demo.cast
# run your devkit commands...
# exit

# Upload and get a sharable link
asciinema upload demo.cast
```

```
# Embed in README.md:  
[![devkit  
demo](https://asciinema.org/a/XXXXX.svg)](https://asciinema.org/a/  
XXXXX)
```

Step 17 — Final demo checklist

Present a live demo covering:

1. devkit gh issues — rich table of open issues
2. devkit gh pr-summary <number> — PR overview with AI review
3. devkit workflow feature-start 'my-feature' --issue 5 — full workflow
4. devkit ai commit — AI-generated commit message from staged diff
5. devkit ai explain 'git rebase -i HEAD~3' — Copilot CLI explanation

Grading

Criterion	Description	Points
Tool integration	CLI tools called as subprocesses	20 pts
Workflow command	feature-start works end-to-end	20 pts
Code quality	Clean structure, typed functions, no spaghetti	10 pts
Error handling	Missing tools, bad IDs, network errors handled	15 pts
UX & rich output	Formatted tables, panels, spinners where needed	10 pts
README & demo	Installable with clear docs and a demo	25 pts

The goal is not just to learn these tools — it's to develop the instinct for composability: seeing two CLI tools and immediately imagining how they could be piped together into something greater than the sum of their parts. That instinct is what separates a developer from a craftsperson.